Microsoft

AI-200T00A

Deploy and manage apps on Azure Container Apps

© Copyright Microsoft Corporation. All rights reserved.

Agenda

- Deploy containers to Azure Container Apps
- Manage containers in Azure Container Apps
- Scale containers in Azure Container Apps

© Copyright Microsoft Corporation. All rights reserved.

Module 1: Deploy containers to Azure Container Apps

© Copyright Microsoft Corporation. All rights reserved.

Learning objectives

- Explain how Azure Container Apps environments affect networking, logging, and isolation
- Deploy a container app using the Azure CLI and using the Azure CLI with a YAML definition
- Configure runtime settings using environment variables and secrets
- Configure image pull authentication for private registries
- Verify container app health using logs, revisions, and replica status

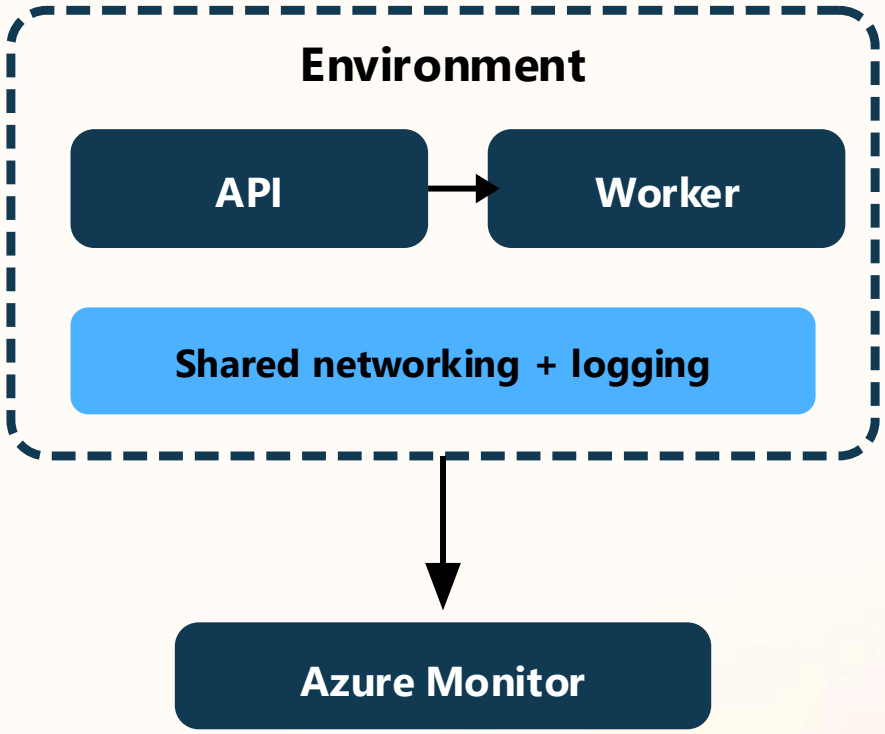
© Copyright Microsoft Corporation. All rights reserved.

Introduction

- Serverless container platform on Azure
- No Kubernetes control plane to manage
- Revision-based deployment model for safe rollouts
- Built-in ingress with external and internal options
- Native secrets and environment variable support
- Integrates with ACR, Azure Monitor, VNET, managed identities

© Copyright Microsoft Corporation. All rights reserved.

Explore Container Apps environments



What an environment provides

- Shared networking boundary for grouped apps
- Internal DNS for app-to-app communication
- Common logging integration (Azure Monitor)

Design decisions

- **Separate by lifecycle:** Dev, test, prod environments
- **Align to networking:** Group apps that communicate privately
- **Plan observability early:** Choose log destination before production

© Copyright Microsoft Corporation. All rights reserved.

Deploy a container app using the Azure CLI and YAML

Three deployment paths

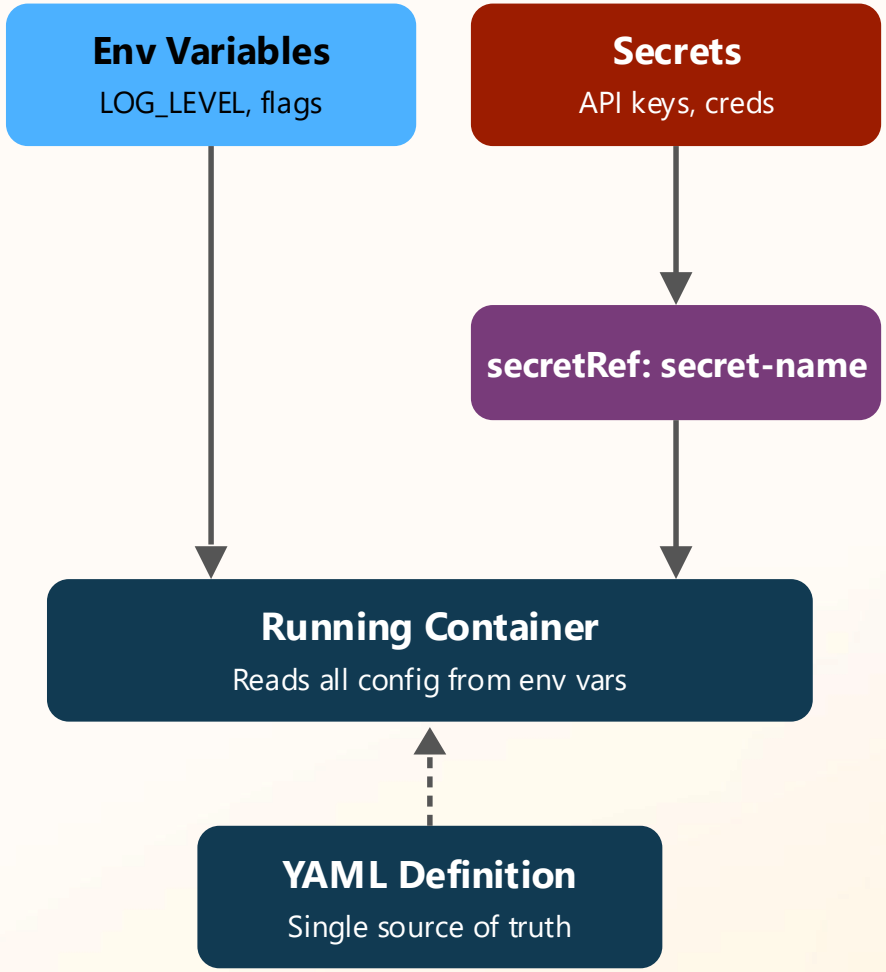
- **`containerapp up:`** Fastest path, creates environment automatically
- **`containerapp create:`** Explicit control, existing environment
- **`--yaml:`** Source-controlled config, reviewable via code review

Revisions

- Each update creates a new revision (versioned snapshot)
- Validate new revision before shifting traffic
- Use `containerapp update` for image and config changes

© Copyright Microsoft Corporation. All rights reserved.

Configure runtime settings with environment variables and secrets



Environment variables (non-sensitive)

- Log levels, feature flags, dependency URLs
- Same image runs in dev, staging, prod with different values
- Set at create time or add incrementally later

Secrets (sensitive values)

- API keys, credentials, signing keys stored separately
- Rotate secrets without rebuilding or redeploying images
- Key Vault references supported for centralized management

Secret references

- Map secrets to env vars so app code reads them transparently
- Keep secret values out of YAML files and shell history

© Copyright Microsoft Corporation. All rights reserved.

Configure image pull authentication for private registries

Username / password

- Works with any registry (ACR, Docker Hub, GHCR)
- Quick to set up for validation and prototyping
- More secrets to manage and rotate over time

Managed identity (recommended for ACR)

- No credentials to store or rotate
- Grant only AcrPull for least privilege
- Best fit for production AI services pulling many images
- Use `--identity system` for ACR

Key consideration: Registry config is separate from app config. Image pull failures often look like auth errors.

© Copyright Microsoft Corporation. All rights reserved.

Verify deployments with logs and status

Logs (fastest signal)

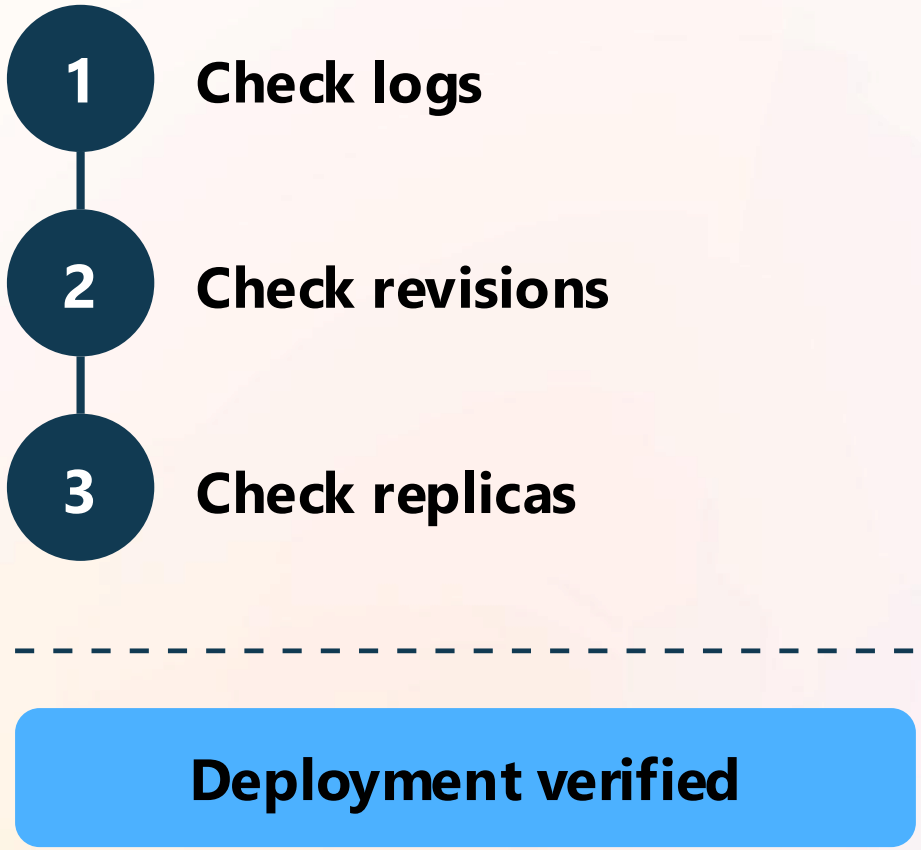
- Console logs surface startup errors, missing env vars, connection failures
- System logs reveal platform-level issues (image pull, scaling)
- Stream logs in real time during active debugging

Revisions (version check)

- Confirm a new revision exists after each update
- Verify the revision reached a healthy, active state
- Include inactive revisions for deeper investigation

Replicas (scaling check)

- Detect scale-to-zero, crash loops, and cold start issues
- Verify expected instance count under load



© Copyright Microsoft Corporation. All rights reserved.

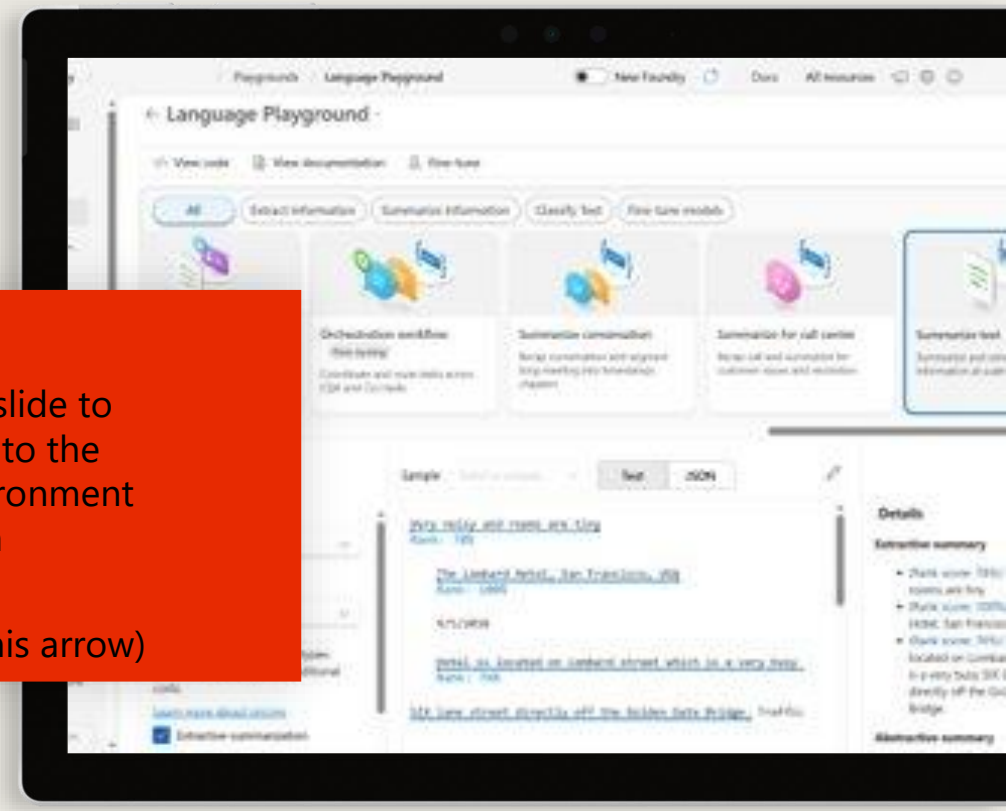
Exercise

Deploy a containerized backend API to Azure Container Apps

In this exercise, you deploy a containerized backend API to Azure Container Apps. You use a managed identity to securely pull images from Azure Container Registry and configure secrets as environment variables.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2348000



Trainers:

Customize this slide to include the link to the hosted lab environment and any custom instructions.

(then remove this arrow)

© Copyright Microsoft Corporation. All rights reserved.

Module 1 knowledge check

1

You want a consistent, source-controlled deployment for a container app so configuration is reviewed like code. Which approach best supports that goal?

A. Use `az containerapp create --yaml` with a YAML file in source control
B. Use `az containerapp update --set-env-vars` for all changes
C. Rely on image tags and redeploy with `az containerapp update --image`

2

A container app fails to start after you update the image. You need fast feedback to diagnose the issue. Which command is the best first step?

A. Run `az containerapp logs show`
B. Run `az containerapp revision list`
C. Run `az containerapp replica list`

© Copyright Microsoft Corporation. All rights reserved.

Module 1 summary

- **Environments:** Shared boundary for networking, logging, and isolation across related apps
- **Deployment paths:** Prototype quickly, then move to YAML for production repeatability
- **Revisions:** Every config change creates a versioned snapshot for safe rollouts
- **Secrets pattern:** Store secrets separately, reference them from environment variables
- **Registry auth:** Prefer managed identity for ACR in production
- **Verification order:** Logs first for fast diagnosis, then revisions, then replicas

© Copyright Microsoft Corporation. All rights reserved.

Module 2: Manage containers in Azure Container Apps

© Copyright Microsoft Corporation. All rights reserved.

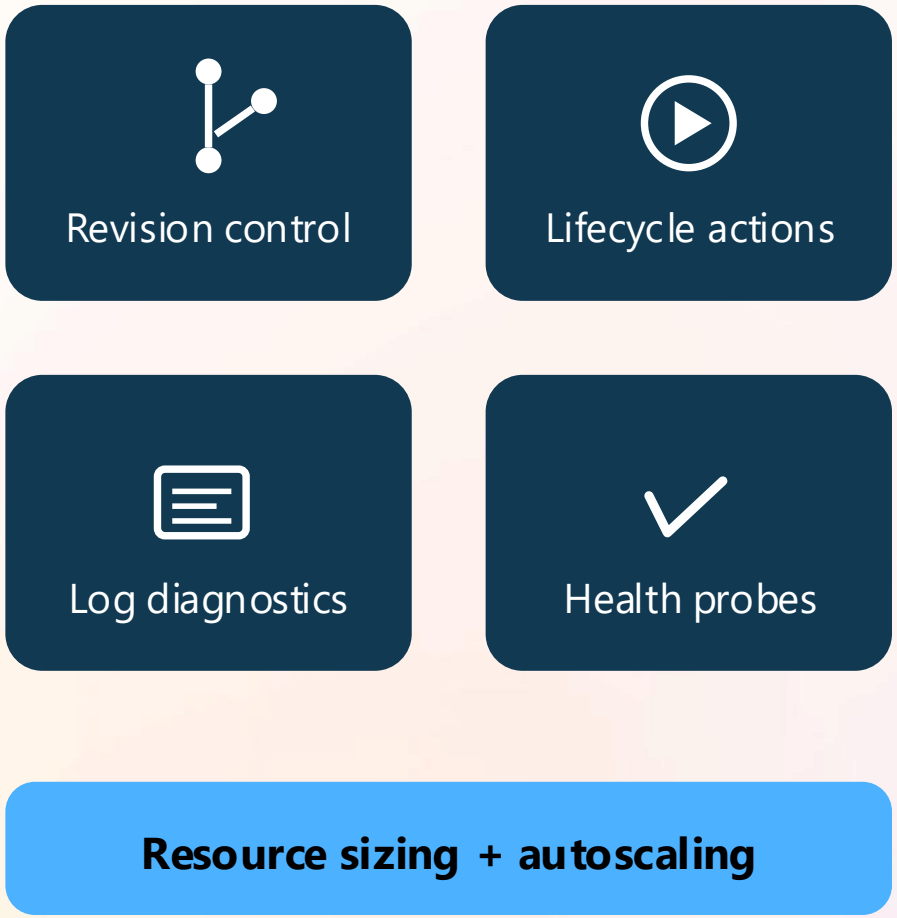
Learning objectives

- Update container images and manage revisions safely during development and release cycles
- Perform app lifecycle operations and diagnose failing revisions
- Monitor logs and troubleshoot common revision and runtime issues
- Configure health probes and troubleshoot probe failures
- Optimize container resources and scaling settings to balance cost and performance

© Copyright Microsoft Corporation. All rights reserved.

Introduction

- Day-two operations for containerized AI workloads
- Revision-based model: every update creates a versioned snapshot
- Built-in lifecycle actions for incident response
- Integrated logging with Azure Monitor
- Health probes gate traffic to healthy replicas
- Per-replica sizing and autoscaling for cost control



© Copyright Microsoft Corporation. All rights reserved.

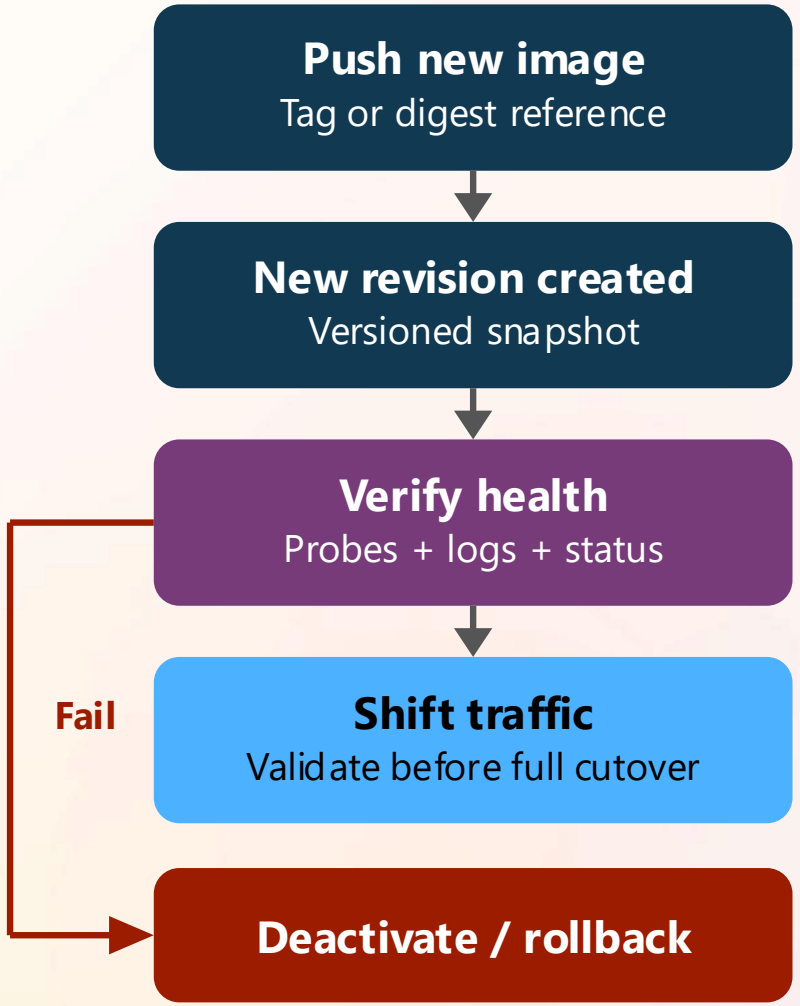
Update images and manage revisions safely

Image identity

- Tags are convenient but mutable. Digests are immutable.
- Use digests in production for traceability during incidents
- Correlate operational behavior with a specific build artifact

Revision workflow

- Each image update creates a new revision
- Verify health before shifting traffic
- Deactivate problematic revisions. Preserve for investigation.
- Define a retention strategy for inactive revisions



© Copyright Microsoft Corporation. All rights reserved.

Manage the container app lifecycle

Choose the right action

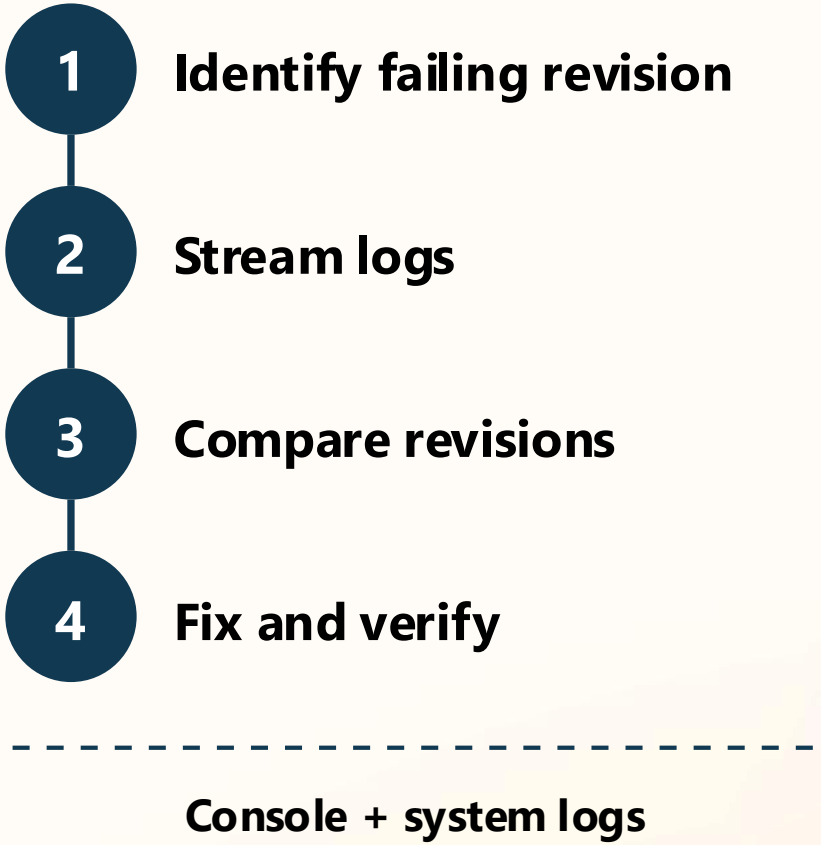
- **Stop:** Guarantees zero traffic. Broad scope. Use during incidents.
- **Restart:** Clears transient state. Use for deadlocks, stuck processes.
- **Deactivate revision:** Isolates one bad release. Safest first step.

Common failure patterns to validate

- Image pull failures (registry credentials)
- Port mismatches (container vs. ingress config)
- Missing environment variables or secrets
- Probe failures (wrong path, short timeout)
- Resource pressure (CPU throttling, OOM restarts)

© Copyright Microsoft Corporation. All rights reserved.

Monitor logs and troubleshoot issues



Log types

- Console logs: Application output (errors, startup messages)
- System logs: Platform events (image pull, scaling, probes)

Repeatable troubleshooting workflow

- Identify which revision is failing
- Stream logs while reproducing the issue
- Compare working vs. failing revision config
- Apply targeted fix, verify next revision is healthy

Structured logging tips

- Include request ID, revision ID, model version
- Log latency breakdowns for dependency calls
- Avoid logging sensitive content (prompts, documents)

© Copyright Microsoft Corporation. All rights reserved.

Configure health probes and troubleshoot failures

Readiness vs. liveness

- Readiness: Guards model loading and warmup.
- Liveness: Catches deadlocks. Triggers restart.

Tuning for AI services

- Set `initialDelaySeconds` to cover model load time
- Use generous `failureThreshold` for slow warmup
- Don’t let liveness fire during startup
- Probe endpoints should not depend on external services

When probes fail immediately after deploy

- Validate port and path match the container’s HTTP server
- Check that timeout allows enough time for response

© Copyright Microsoft Corporation. All rights reserved.

Optimize container resources and scaling

Per-replica sizing

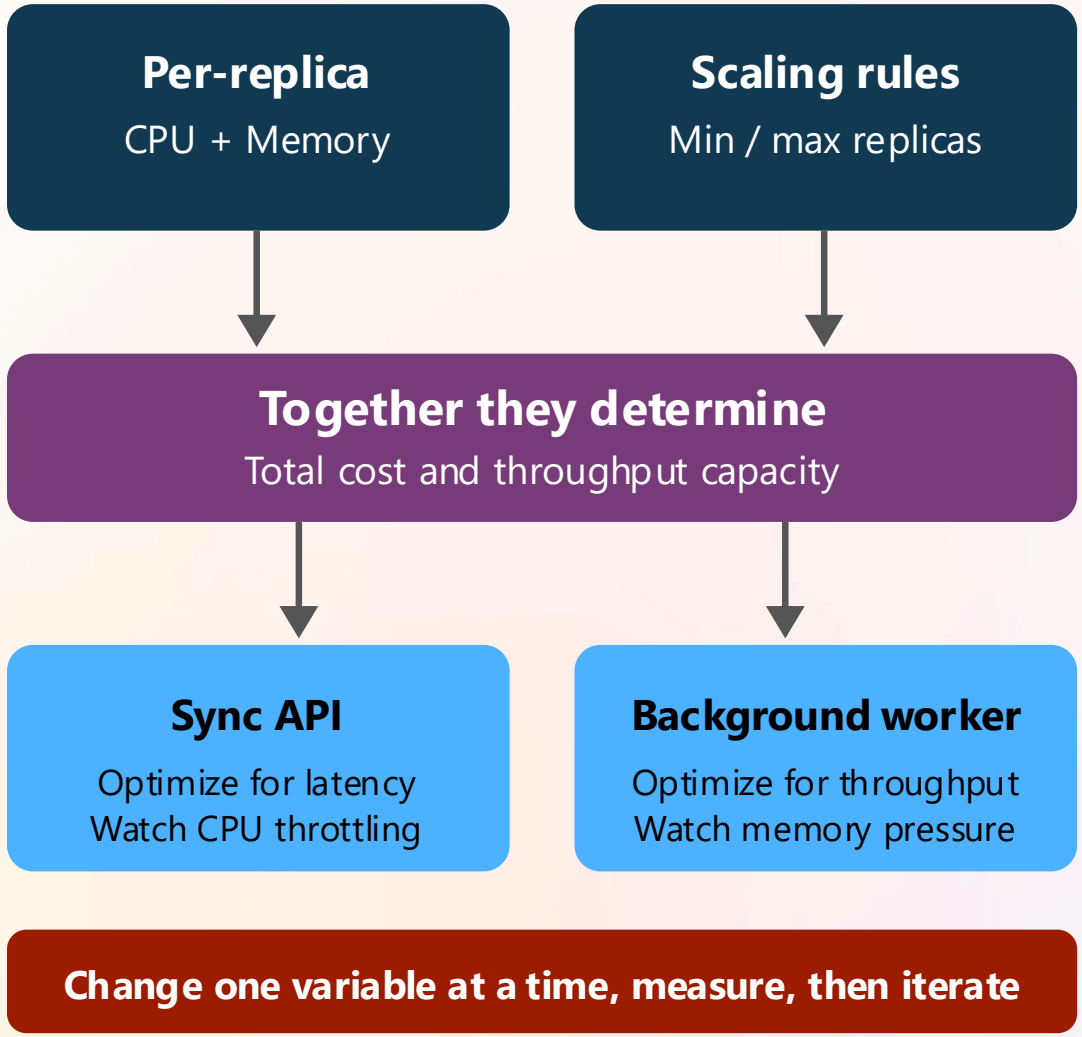
- CPU throttling = need more CPU per replica
- OOM restarts = need more memory per replica
- Total cost = per-replica size x replica count

Scaling strategies differ by workload

- **Sync API:** Minimum replicas to reduce cold starts. Optimize for latency.
- **Background worker:** Scale-to-zero if event-driven. Optimize for throughput.

Optimization approach

- Measure before changing (logs + metrics)
- Change one variable at a time
- Reassess after model version changes



© Copyright Microsoft Corporation. All rights reserved.

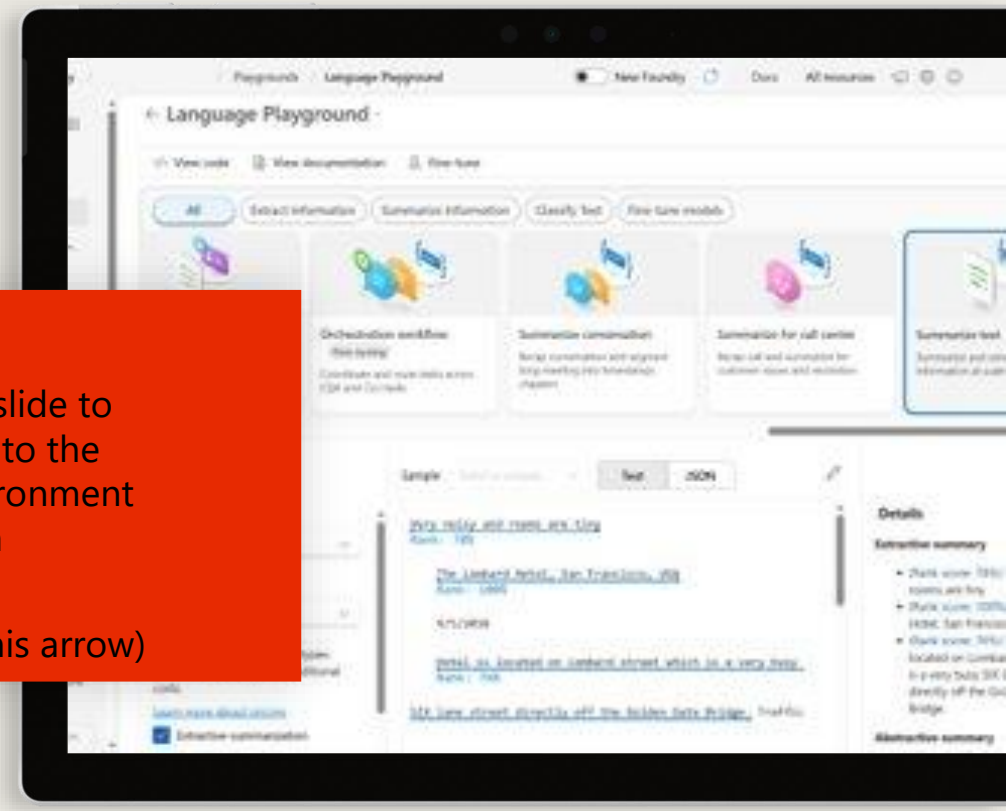
Exercise

Diagnose and fix a failing deployment

In this exercise, you troubleshoot a failing container app and apply targeted fixes. You use revision status, logs, and the Azure CLI to isolate deployment issues.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2348400



Trainers:

Customize this slide to include the link to the hosted lab environment and any custom instructions.

(then remove this arrow)

© Copyright Microsoft Corporation. All rights reserved.

Module 2 knowledge check

1 You deploy a new image to a production container app. Which approach provides the best traceability and reduces the risk of deploying the wrong artifact?

A. Reference the image by digest
B. Reference the image by the `latest` tag
C. Rebuild the image locally in each environment

2 Your AI API shows high latency and log messages indicate CPU throttling during peak traffic. Which change most directly addresses the throttling?

A. Increase the per-replica CPU allocation, then reassess scaling rules
B. Deactivate the newest revision to reduce load
C. Switch the image reference from digest to tag

© Copyright Microsoft Corporation. All rights reserved.

Module 2 summary

- **Image digests:** Immutable references for production traceability. Tags are for convenience only.
- **Revision workflow:** Update, verify health, then shift traffic. Deactivate before deleting.
- **Lifecycle scope:** Revision deactivation isolates one release. Stop and restart affect the whole app.
- **Logs first:** Console and system logs are the fastest diagnostic signal in a managed platform.
- **Probes protect rollouts:** Readiness gates traffic. Liveness triggers restarts. Tune both for real startup behavior.
- **Size then scale:** Fix per-replica bottlenecks before adding replicas. Change one variable at a time.

© Copyright Microsoft Corporation. All rights reserved.

Module 3: Scale containers in Azure Container Apps

© Copyright Microsoft Corporation. All rights reserved.

Learning objectives

- Configure HTTP, TCP, CPU, and memory scale rules for container apps
- Implement event-driven scaling using KEDA scalers for Azure services
- Select appropriate compute resources to optimize performance and cost
- Apply revision modes to control scaling behavior and traffic distribution

© Copyright Microsoft Corporation. All rights reserved.

Introduction

- Automatic horizontal scaling based on demand
- Powered by KEDA (Kubernetes Event-Driven Autoscaling)
- Scale to zero for cost savings during idle periods
- Built-in HTTP, TCP, CPU, and memory triggers
- Event-driven scaling for queues, streams, and custom metrics
- Traffic splitting across revisions for safe rollouts

© Copyright Microsoft Corporation. All rights reserved.

Configure scale rules

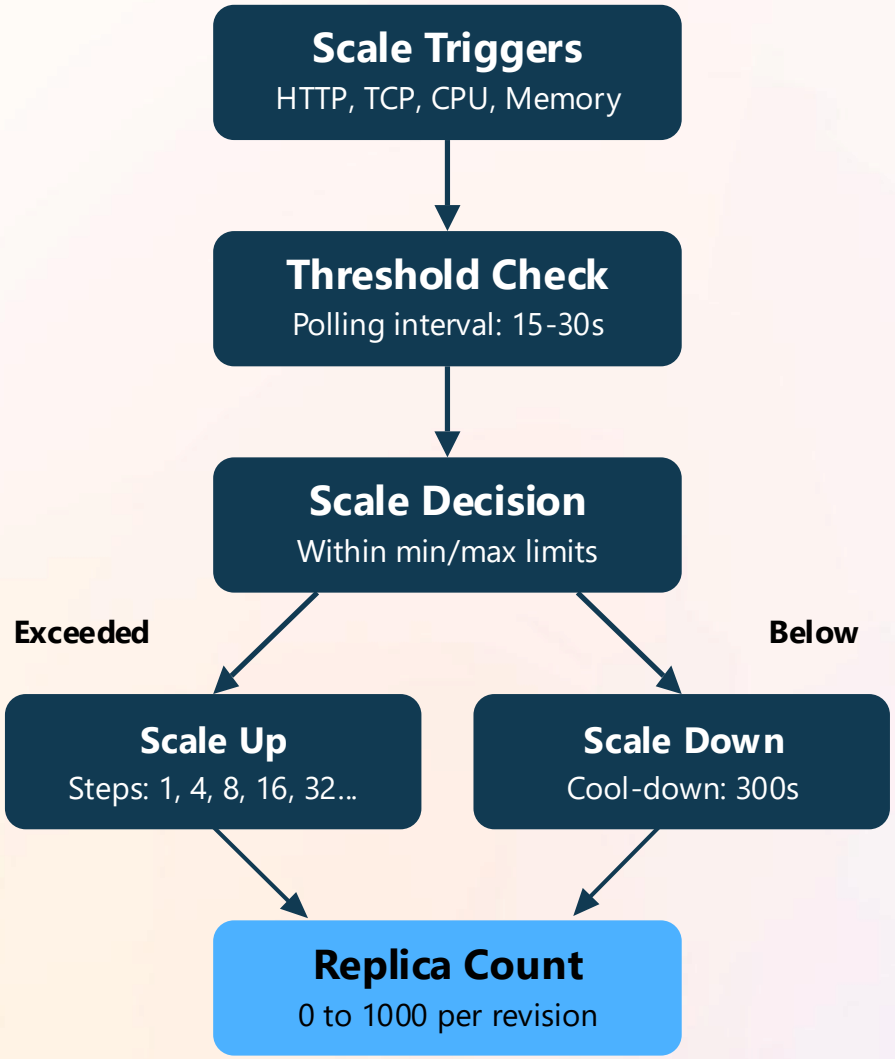
Scale definition components

- Limits: Min and max replica count (0 to 1000)
- Rules: Triggers that drive scaling decisions
- Behavior: Timing for polling, cool-down, stabilization

Built-in rule types

- HTTP: Concurrent requests per replica (default: 10)
- TCP: Concurrent connections for persistent protocols
- CPU: Utilization percentage (cannot scale to zero)
- Memory: Consumption percentage (cannot scale to zero)

Key behavior: Multiple rules use highest replica count



© Copyright Microsoft Corporation. All rights reserved.

Implement event-driven scaling with KEDA

Azure-native scalers (first-party)

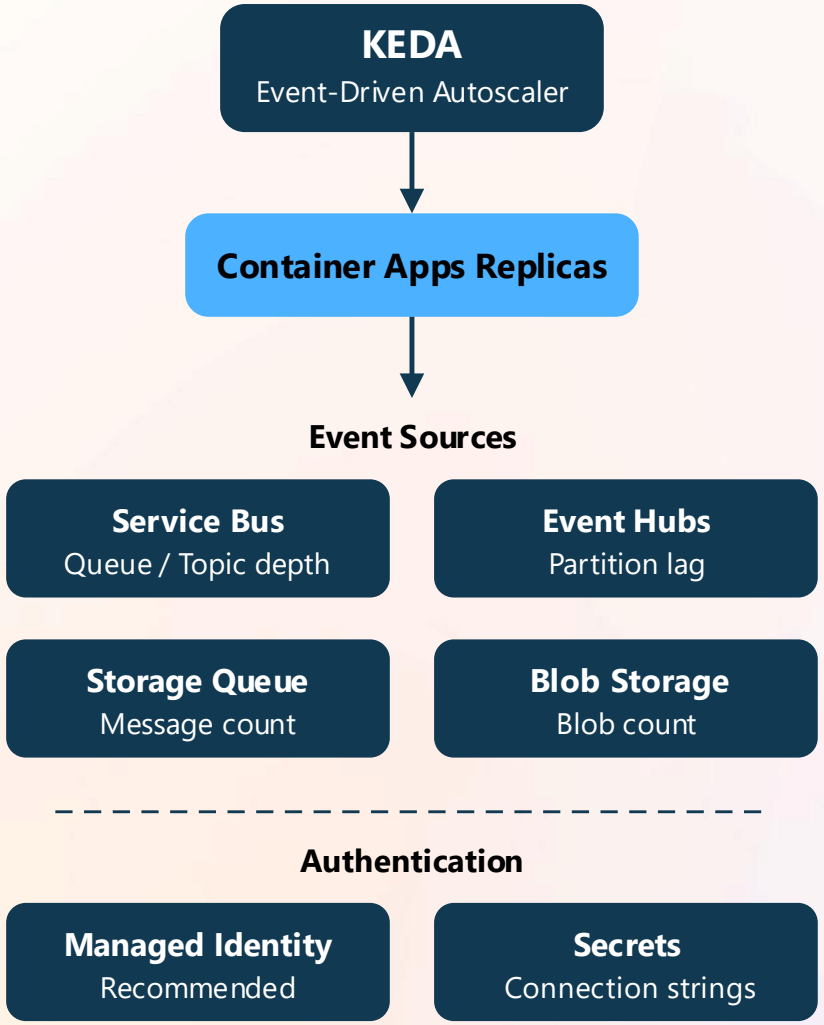
- Service Bus: Scale on queue/topic message count
- Event Hubs: Scale on partition consumer lag
- Storage Queue: Scale on approximate message count
- Blob Storage: Scale on blob count

How it works

- KEDA polls event source every 30 seconds
- Divides current metric by threshold to get replica count
- Supports scale-to-zero for idle queue processors

Authentication

- Managed identity (recommended for production)
- Secrets-based connection strings



© Copyright Microsoft Corporation. All rights reserved.

Apply KEDA scalers for custom workloads

Beyond Azure-native scalers

- Kafka: Consumer group lag per partition
- Redis: List length or stream pending entries
- Cron: Time-based pre-scaling for known peaks
- Prometheus: Custom PromQL business metrics

Combining scalers

- Cron establishes baseline capacity on schedule
- HTTP or event scalers handle real-time demand
- Platform uses highest replica count across all rules

Migrating from Kubernetes KEDA

- Map ScaledObject type to Container Apps rule type
- Extract metadata and auth parameters
- Replace TriggerAuthentication with secrets or identity

© Copyright Microsoft Corporation. All rights reserved.

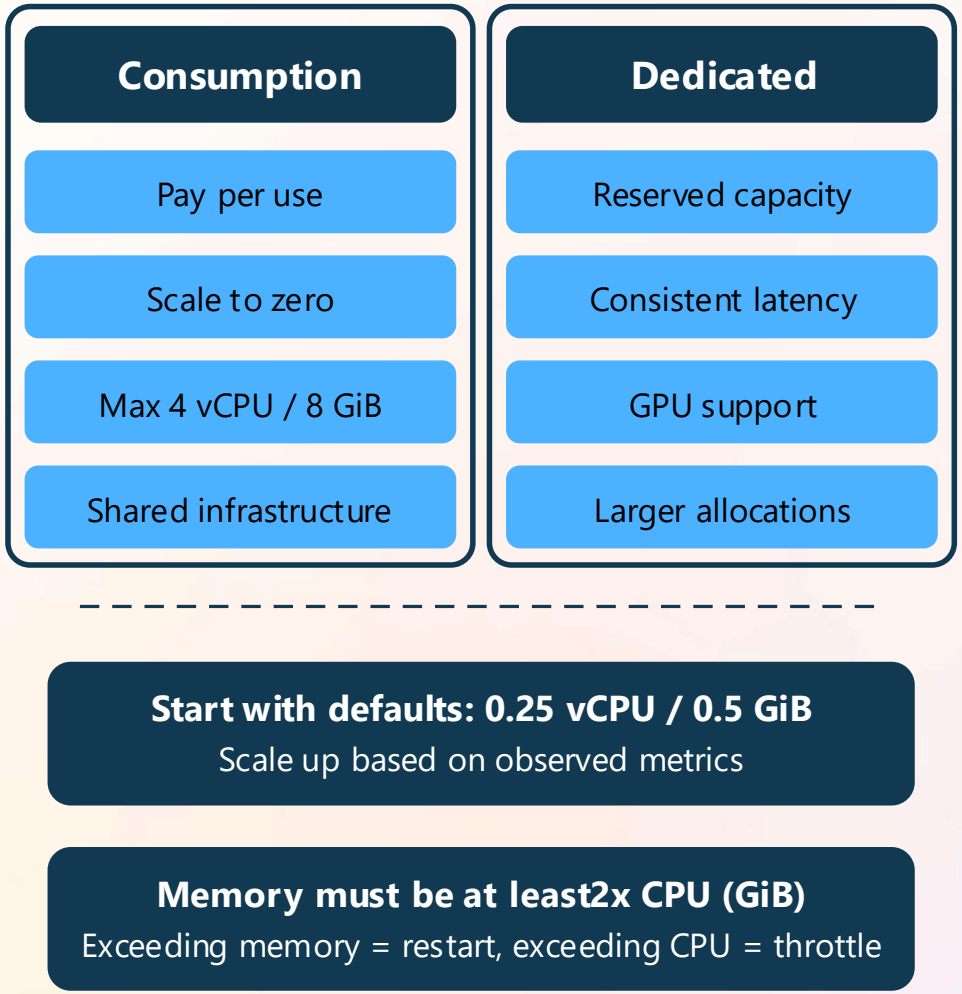
Select compute resources for performance and cost

Resource allocation

- CPU in cores, memory in GiB per container
- Memory must be at least 2x CPU value
- Exceeding memory limit restarts the replica
- Exceeding CPU limit throttles (no restart)

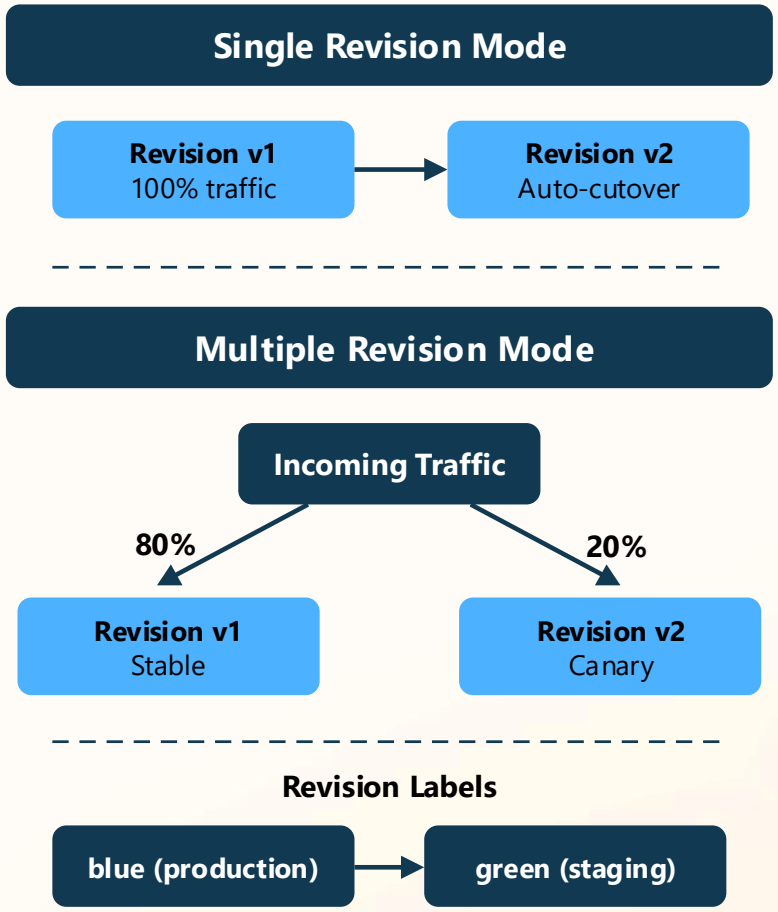
Cost levers

- Scale-to-zero eliminates idle compute charges
- Right-size based on actual utilization, not estimates
- Total capacity = per-replica resources x max replicas
- Smaller replicas scale more precisely but more frequently



© Copyright Microsoft Corporation. All rights reserved.

Choose and apply revision modes



Single revision mode (default)

- One active revision at a time
- Zero-downtime deployment is automatic
- New revision replaces old after health checks pass
- No traffic splitting capability

Multiple revision mode

- Multiple revisions active simultaneously
- Traffic splitting by percentage weights
- Canary releases, blue-green deployments, A/B testing
- Must deactivate old revisions manually

Revision labels

- Named endpoints for direct revision access
- Test specific revisions without affecting production traffic
- Enable blue-green switching via label moves

© Copyright Microsoft Corporation. All rights reserved.

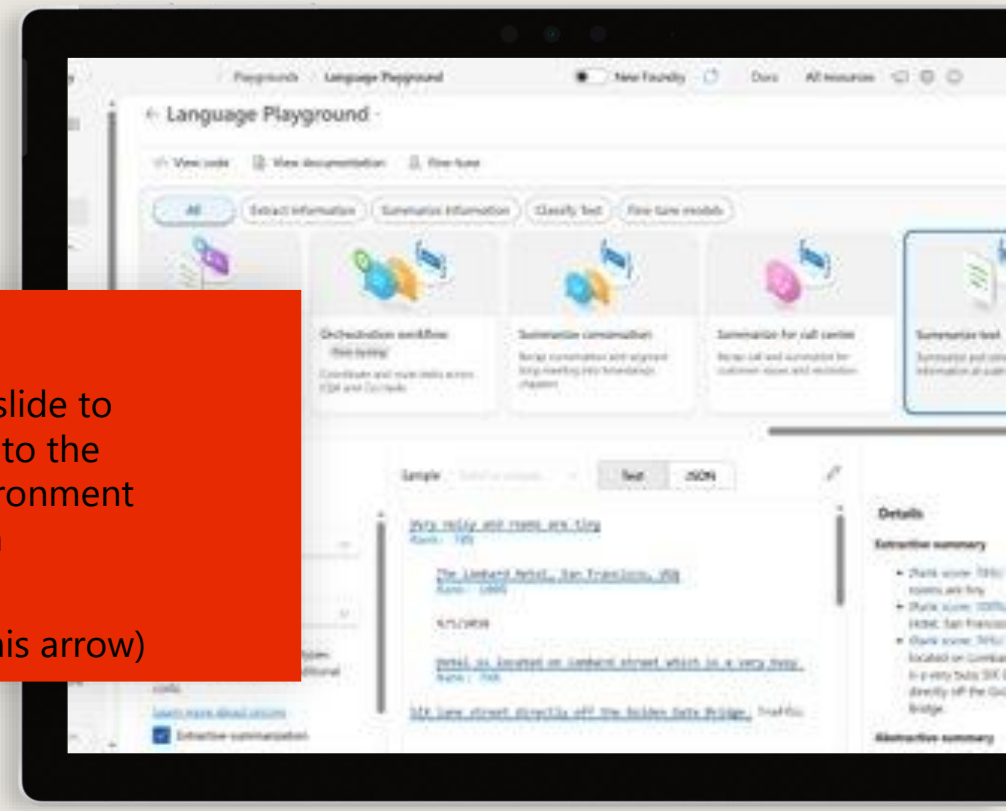
Exercise

Configure autoscaling using KEDA triggers

In this exercise, you deploy a simple mock agent API and configure autoscaling based on HTTP concurrent requests.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2348401



Trainers:

Customize this slide to include the link to the hosted lab environment and any custom instructions.

(then remove this arrow)

© Copyright Microsoft Corporation. All rights reserved.

Module 3 knowledge check

1 A developer wants to configure a container app that processes messages from an Azure Service Bus queue and scales to zero replicas when no messages are present. Which scaling configuration meets this requirement?

A. Set `--min-replicas 0` with an HTTP scale rule
B. Set `--min-replicas 0` with a CPU scale rule
C. Set `--min-replicas 0` with an azure-servicebus scale rule

2 A development team wants to ensure their container app has five replicas ready before the morning traffic spike at 8 AM, while still allowing scale-to-zero overnight. Which scaling approach meets this requirement?

A. Configure a cron scale rule combined with an HTTP scale rule
B. Set minimum replicas to five with an HTTP scale rule
C. Configure a CPU scale rule with a low utilization threshold

© Copyright Microsoft Corporation. All rights reserved.

Module 3 summary

- **Index types:** range for filters, composite for multi-property sort, vector for similarity, spatial for geo queries
- **Composite indexes** must match exact property sequence and sort direction in ORDER BY clauses
- **Vector index selection:** flat for small datasets, quantizedFlat for medium, diskANN for large-scale production
- **Selective indexing** excludes unused properties. Always exclude embedding arrays from range indexes
- **Query metrics** reveal missing indexes through low utilization and high retrieved-to-output ratios
- **Session consistency** provides read-your-writes for user-facing operations at 1x RU cost

© Copyright Microsoft Corporation. All rights reserved.

Group discussion

© Copyright Microsoft Corporation. All rights reserved.

Group discussion questions

- When you deploy a backend to Azure Container Apps, how do you decide which settings need to be designed up front, such as environment boundaries, secret handling, registry access, and runtime configuration, versus which settings you can refine after the first release?

- Azure Container Apps makes it easier to roll out updates with revisions, logs, and health probes, but day-two operations still require trade-offs. How do you balance fast iteration and safe releases when a new container image might improve features but also introduce runtime or probe failures?

- AI workloads often have unpredictable traffic patterns, background jobs, and changing latency targets. How do you choose between simple HTTP-based autoscaling, event-driven scaling with KEDA, and more deliberate traffic management strategies so the application stays responsive without overpaying for compute?

© Copyright Microsoft Corporation. All rights reserved.

End of presentation

© Copyright Microsoft Corporation. All rights reserved.